Secret authority
Vault
Project source credentials
Project encryption key

Deployment pipeline
GitLab CI
Reads project Vault paths
Applies Kubernetes Secrets

Database provisioning
Project DB + LOGIN
Creates database, role and password
Delivers authorizer-db Secret

External provider
OAuth authentication endpoint
HTTPS / client_credentials
Returns a derived access token

Kubernetes cluster

namespace: auth-agent-a

Secret / application-credentials
Source credentials
/credentials/credentials.json
OAuth client ID + client secret

Secret / auth-state-key
Encryption key K_a
/auth-state-key/key
Base64 of 32 random bytes

Secret / authorizer-db
DATABASE_URL
Database: auth_agent_a
Login: auth_agent_a + password

Read-only file mounts → authorizer

Environment → authorizer

Deployment/agent-egress

replicas: G_a

egress Pod 1

Container
authorizer
OAuth resolver + pkg/authstate
On token use: SQL recheck after 1 s
One pgx pool: ≤ 4 SQL connections

Envoy: inject derived bearer token

egress Pod N

Container
authorizer
OAuth resolver + pkg/authstate
On token use: SQL recheck after 1 s
One pgx pool: ≤ 4 SQL connections

Envoy: inject derived bearer token

SQL / TLS

Same project database

HTTPS · refresh lease holder only

Restart: decrypt and reuse valid shared state.
Source credentials and K_a stay in mounted files.

namespace: auth-agent-b

Project Secrets
Independent credentials + K_β
Own OAuth client credentials
Own 32-byte encryption key
Own DATABASE_URL + password

authorizer replicas × G□
Project B proxy bundle
Same K_β within project B
Login / database: auth_agent_b
Own local caches and connection pools

SQL / TLS

Project replicas share auth state. Projects have separate databases, logins and encryption keys.

namespace: kagent

PostgreSQL server

kagent-postgresql.kagent.svc:5432
Verified TLS + SCRAM-SHA-256 password authentication

database: auth_agent_a

LOGIN / OWNER: auth_agent_a

auth_state.tokens

id	text	PK / HMAC context ID
version	bigint	NOT NULL
ciphertext	bytea	NULL until acquired
expires_at	timestamptz	NOT NULL / safe deadline
owner	text	NOT NULL
lease_until	timestamptz	NOT NULL
retry_after	timestamptz	NOT NULL

ciphertext = nonce + AES-256-GCM({value, expires}); AAD = id/version

auth_bridge.bindings — Per-request identity / revocation

session_id	text	PRIMARY KEY
data	jsonb	NOT NULL
active	boolean	NOT NULL
created_at	timestamptz	NOT NULL
revoked_at	timestamptz	NULL allowed

INDEX bindings_gc (active, created_at)

auth_bridge.launches — Creation retries → same Session

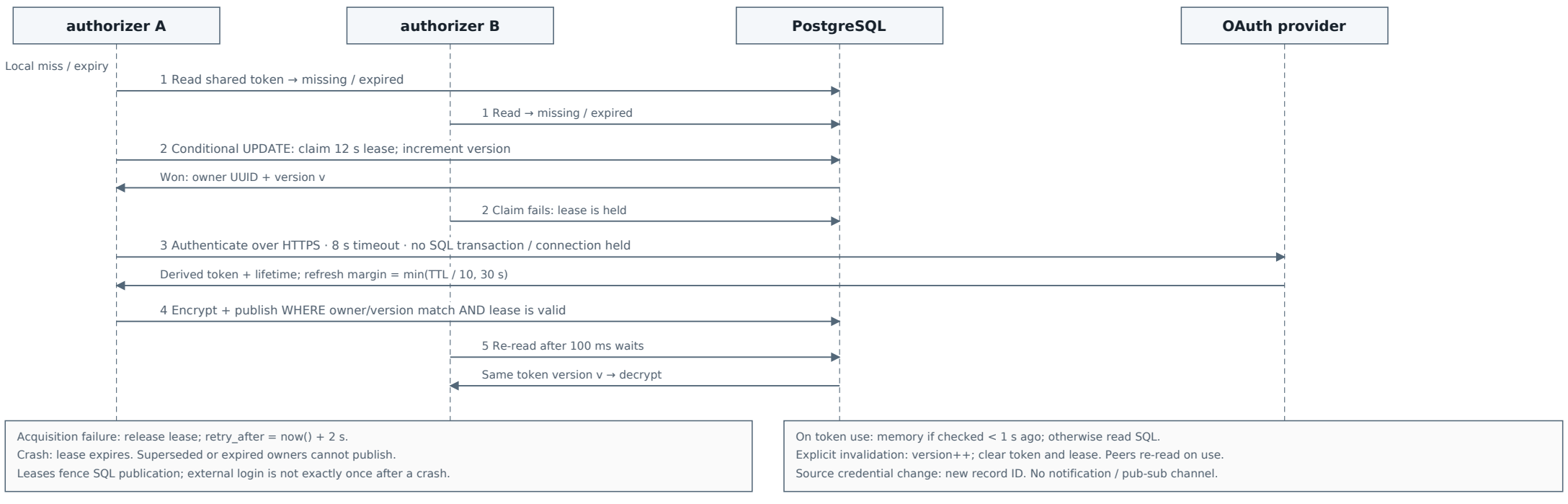
caller	text	NOT NULL
request_id	text	NOT NULL
request	jsonb	NOT NULL
account_uid	text	NOT NULL
binding	jsonb	NULL allowed
created_at	timestamptz	NOT NULL
retired_at	timestamptz	NULL allowed

PRIMARY KEY (caller, request_id)

Database / auth_agent_b
LOGIN / OWNER: auth_agent_b
Own auth_state.tokens + auth_bridge.bindings / launches

Platform databases
kagent / Substrate
Separate platform tables and roles

Refresh: same project + credential configuration



Acquisition failure: release lease; retry_after = now() + 2 s.
Crash: lease expires. Superseded or expired owners cannot publish.
Leases fence SQL publication; external login is not exactly once after a crash.

On token use: memory if checked < 1 s ago; otherwise read SQL.
Explicit invalidation: version++; clear token and lease. Peers re-read on use.
Source credential change: new record ID. No notification / pub-sub channel.